Empirical Investigation of Formatting Drift: Parameter Scale and Post-Training Quantization Dynamics in Small Language Models

Vedartha Sai Y
Indian Institute of Information Technology, Allahabad (IIITA)
Department of Computer Science and Engineering [TO VERIFY]
Allahabad, India
Email: vedartha.sai@example.com [TO VERIFY]

Abstract—As Large Language Models (LLMs) are increasingly integrated into autonomous software pipelines requiring structured machine-readable outputs (e.g., JSON), maintaining schema adherence under computational optimization constraints is critical. This study empirically quantifies Formatting Drift ($D_{\text{formatting}}$)—the systematic regression in structured schema compliance—under post-training dynamic INT8 quantization across parameter scales (0.5B vs. 1.5B parameters) using the Qwen2.5-Instruct model family. Evaluating $N = 100$ stratified benchmark prompts across three schema complexity tiers under deterministic greedy decoding ($T = 0.0$, seed = 42), we refute the Parameter Capacity Buffering Hypothesis. While scaling from 0.5B to 1.5B parameters achieves 100.0% unquantized FP16 semantic extraction accuracy, it fails to buffer against dynamic INT8 structural collapse, yielding severe formatting drift ($D_{\text{formatting}} = -93.0\%$ for 0.5B and $D_{\text{formatting}} = -100.0\%$ for 1.5B; $p < 10^{-15}$). Qualitative failure analysis reveals that dynamic INT8 quantization induces attention routing collapse, system prompt role erosion, and schema definition echoing (31.6% of failures). These findings demonstrate that structural schema adherence is extraordinarily fragile under weight precision reduction, requiring specialized decoding constraints rather than passive parameter capacity scaling.

Index Terms—Large Language Models, Formatting Drift, Quantization, Schema Adherence, Model Compression, Structural Robustness.

I. Introduction

Modern software architectures frequently rely on Large Language Models (LLMs) to function as natural-language-to-structured-data translation engines. In autonomous multi-agent systems and enterprise database pipelines, model completions must adhere strictly to formal machine-readable specifications, such as Draft-07 JSON Schema. A single syntax violation (e.g., an unclosed bracket or conversational preamble) invalidates the output stream, causing runtime execution exceptions in downstream parsers.

To lower operational latency and hardware memory footprints during local CPU deployment, post-training weight quantization (e.g., converting 16-bit floating-point weights to 8-bit dynamic integers) is widely applied. While conventional benchmark evaluations assess general task accuracy, reasoning benchmarks, or perplexity under quantization, structural schema compliance represents a distinct and rigid operational constraint.

In this work, we formalize and measure Formatting Drift ($D_{\text{formatting}}$), defined as the relative percentage change in schema compliance pass rate between an unquantized baseline model and its quantized variant under identical decoding parameters.

We address three primary research questions:

- RQ_1 (Complexity Nonlinearity): Is formatting drift ($D_{\text{formatting}}$) non-linearly amplified as target schema complexity increases from flat key-value pairs to nested hierarchies and array sequences?
- RQ_2 (Parameter Capacity Buffering Hypothesis): Does scaling parameter capacity (0.5B $\rightarrow$ 1.5B) buffer against post-training quantization formatting drift under identical execution parameters?
- RQ_3 (Surface vs. Structural Degradation): What proportion of quantization drift stems from surface markdown code fence syntax additions versus inner schema structural corruption (SYNTAX_ERROR / SCHEMA_FIELD_MISMATCH)?

Through a controlled 2×2 comparative matrix evaluating Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct across unquantized FP16 and dynamic INT8 precisions on $N = 100$ stratified prompt schemas, we demonstrate that dynamic quantization severely degrades schema compliance ($p < 10^{-15}$), and that scaling parameter capacity fails to mitigate structural failure.

II. Related Work

A. Structured LLM Generation and Schema Adherence

Ensuring LLMs adhere to structured output constraints has evolved from prompt engineering to programmatic decoding interventions. Early techniques relied on system prompt instructions and few-shot examples to encourage valid JSON generation. However, unconstrained autoregressive decoding remains susceptible to syntax violations and conversational chatter. Recent approaches utilize grammar-constrained decoding (e.g., Outlines, JSONLogitsProcessor) to mask invalid tokens during inference. Our

work evaluates the intrinsic capacity of instruction-tuned models to adhere to schema constraints without external logit masking interventions.

B. Post-Training Quantization and Weight Precision

Post-training quantization (PTQ) compresses neural network parameters to low-bit representations (e.g., INT8, INT4) to accelerate inference and reduce peak memory consumption. While PTQ techniques maintain competitive performance on broad perplexity benchmarks, recent studies indicate that outlier activations in transformer projection layers can be severely distorted under low-precision quantization. This work investigates how dynamic quantization impacts specialized structural control flow within small instruction-tuned models.

C. Parameter Scaling and Robustness

The parameter scaling hypothesis posits that increasing model scale confers resilience against input noise and capacity loss from compression. Larger models develop redundant representations that buffer against precision reduction. We explicitly test whether scaling from 0.5B to 1.5B parameters provides structural buffering against quantization-induced formatting collapse.

III. Methodology

A. Formatting Drift Formalization

Let $P_{\text{base}} \in [0, 1]$ denote the schema compliance pass rate of an unquantized baseline model (FP16), and $P_{\text{quant}} \in [0, 1]$ denote the compliance pass rate of the quantized variant (INT8). Relative Formatting Drift ($D_{\text{formatting}}$) is defined as:

$$D_{\text{formatting}} = \frac{P_{\text{quant}} - P_{\text{base}}}{P_{\text{base}}} \times 100\% \tag{1}$$

B. Dual-Pass Verification Engine

To eliminate LLM-as-a-judge bias, model completions are evaluated deterministically using Python jsonschema Draft-07 validators:

1) Pass 1 (Direct Raw Pass, P_{raw}): Evaluates the exact completion string emitted by the model without post-processing.
2) Pass 2 (Scrubbed Pass, P_{scrubbed}): Applies regular expressions to extract JSON payloads wrapped within markdown code blocks (“‘json ... “‘) prior to schema validation.

Outputs failing both verification passes are assigned to a mutually exclusive failure category:

- SYNTAX_ERROR: Unparseable JSON syntax, unclosed brackets, conversational preambles, or truncated text.
- SCHEMA_FIELD_MISMATCH: Valid JSON syntax that omits mandatory keys declared in the target schema.
- SCHEMA_TYPE_MISMATCH: Valid JSON syntax containing required keys, but with incorrect value data types.

C. Stratified Benchmark Dataset

The evaluation benchmark comprises $N = 100$ context-schema pairs partitioned into three complexity tiers:

- Tier 1: Primitive Flat ($N_1 = 35$): Flat key-value schemas containing basic primitive data types (string, integer, boolean).
- Tier 2: Nested Object ($N_2 = 35$): Hierarchical structures containing multi-level nested sub-objects.
- Tier 3: Typed Array ($N_3 = 30$): Homogeneous and heterogeneous array structures requiring sequential structure tracking.

D. Statistical Significance Testing

To determine if observed pass rate deltas are statistically significant, we employ a two-proportion pooled Z-test at significance level $\alpha = 0.05$:

$$Z = \frac{P_{\text{quant}} - P_{\text{base}}}{\sqrt{P_{\text{pool}}(1 - P_{\text{pool}})\left(\frac{2}{N}\right)}}, \quad P_{\text{pool}} = \frac{P_{\text{base}} + P_{\text{quant}}}{2} \tag{2}$$

IV. Experimental Setup

A. Execution Environment and Hardware

All evaluations were executed locally on a Windows CPU workstation equipped with 15.44 GB system RAM and PyTorch native runtime.

B. Model Selection and Quantization Control

We evaluate two instruction-tuned models from the Qwen2.5 series:

1) Qwen/Qwen2.5-0.5B-Instruct (0.49B parameters)
2) Qwen/Qwen2.5-1.5B-Instruct (1.54B parameters)

Quantization is applied to dynamic linear layers using PyTorch dynamic quantization (torch.ao.quantization.quantize_dynamic) converting torch.nn.Linear weights to qint8.

C. Decoding Hyperparameters

Sampling randomness was eliminated by locking greedy decoding hyperparameters across all experimental cells:

- Temperature ($T = 0.0$), Top-p (1.0), do_sample = False.
- Deterministic Random Seed: Locked at seed = 42 across PyTorch, NumPy, and Python.
- System Prompt: Fixed instruction enforcing JSON generation without markdown fencing or preambles.

V. Results and Discussion

A. Full 2×2 Experimental Matrix

Table I summarizes the primary evaluation results across parameter scales and precisions over the $N = 100$ benchmark dataset.

Both baseline models achieved 100.0% structural compliance in unquantized FP16 precision. Applying dynamic INT8 quantization induced severe performance drops, yielding $D_{\text{formatting}} = -93.0\%$ for 0.5B ($Z =$

TABLE I
Full 2×2 Experimental Matrix Results ($N = 100$, Seed=42, $T = 0.0$)

Model Scale	Precision	Raw Pass (P_{raw})	Scrubbed Pass (P_{scrubbed})	Semantic Acc (P_{semantic})	$D_{\text{formatting}}$	Z-Statistic	p-value
Qwen2.5-0.5B	FP16 (Base)	100.0%	100.0%	94.0%	0.0%	—	—
Qwen2.5-0.5B	Dynamic INT8	0.0%	7.0%	0.0%	-93.0%	-13.1845	$< 10^{-15}$
Qwen2.5-1.5B	FP16 (Base)	100.0%	100.0%	100.0%	0.0%	—	—
Qwen2.5-1.5B	Dynamic INT8	0.0%	0.0%	0.0%	-100.0%	-14.1421	$< 10^{-15}$

$-13.1845, p < 10^{-15}$) and $D_{\text{formatting}} = -100.0\%$ for 1.5B ($Z = -14.1421, p < 10^{-15}$).

B. Stratified Tier Breakdown (RQ_1)

Table II breaks down the scrubbed pass rate across schema complexity tiers.

TABLE II
Stratified Scrubbed Pass Rate (P_{scrubbed}) Across Schema Complexity Tiers

Schema Complexity Tier	N	FP16 (Both)	0.5B-INT8	1.5B-INT8
Tier 1: Primitive Flat	35	100.0%	20.0%	0.0%
Tier 2: Nested Object	35	100.0%	0.0%	0.0%
Tier 3: Typed Array	30	100.0%	0.0%	0.0%

While Qwen2.5-0.5B-INT8 retained 20.0% compliance on flat primitive key-value pairs (Tier 1), compliance dropped to 0.0% on nested objects (Tier 2) and typed arrays (Tier 3). Qwen2.5-1.5B-INT8 collapsed to 0.0% across all tiers, confirming that structural complexity non-linearly accelerates formatting failure.

C. Refutation of Parameter Capacity Buffering (RQ_2)

In unquantized FP16 precision, scaling parameters from 0.5B to 1.5B improved semantic extraction accuracy from 94.0% to 100.0%. Under dynamic INT8 quantization, however, the 1.5B model suffered complete structural collapse ($P_{\text{scrubbed}} = 0.0\%$), performing worse than the 0.5B model (7.0%). This finding refutes the hypothesis that parameter capacity scaling buffers against quantization-induced formatting drift.

D. Qualitative Failure Diagnostics (RQ_3)

Categorization of $N = 193$ total failure outputs across INT8 runs revealed two dominant failure modes:

1) Syntax Errors (67.9%, 131/193): Unparseable outputs dominated by conversational preambles (e.g., Assistant:, Question:... Answer:...), markdown bullet lists, stray HTML artifacts (</pre>), and truncated token sequences ('JSON:').
2) Schema Field Mismatches (31.6%, 61/193): Valid JSON outputs that echoed the JSON Schema definition template rather than populating extracted data instances.
3) Schema Type Mismatches (0.5%, 1/193): Valid JSON instances containing required keys but with invalid value data types.

Qualitative inspection indicates that uncalibrated dynamic quantization distorts linear layer projections, leading to attention routing collapse and system prompt role erosion.

VI. Limitations

To preserve analytical rigor, key methodological boundaries are noted:

1) Quantization Method Scope: Evaluation was limited to PyTorch dynamic CPU quantization (torch.ao.quantization.quantize_dynamic). Findings should not be extrapolated to static block-wise quantization formats (e.g., GGUF Q4_K_M) or quantization-aware training (QAT).
2) Sample Size and Diversity: The evaluation benchmark comprised $N = 100$ prompts evaluated on two models within the Qwen2.5-Instruct family.
3) Metric Scope: Pass rates ($P_{\text{raw}}, P_{\text{scrubbed}}$) evaluate JSON schema compliance and do not measure broader factual reasoning capabilities.

VII. Conclusion and Future Work

This study demonstrates that structural schema compliance in small instruction-tuned LLMs is highly fragile under post-training weight quantization. Dynamic INT8 quantization induces catastrophic formatting drift ($D_{\text{formatting}} \leq -93.0\%, p < 10^{-15}$) across sub-2B parameter models. Furthermore, parameter scaling (0.5B $\rightarrow$ 1.5B) fails to buffer against structural collapse.

We recommend three prioritized follow-up directions:

1) GGUF/Block-Wise Quantization Benchmark: Evaluate static calibrated formats (GGUF Q8_0, Q4_K_M) to isolate dynamic quantization artifacts.
2) Grammar-Constrained Decoding Intervention: Test whether logit-masking engines (e.g., Outlines) restore 100% compliance under INT8 without latency overhead.
3) Quantization-Aware Training (QAT): Investigate whether fine-tuning models under quantization constraints prevents attention routing collapse.

Acknowledgments

[TO VERIFY: Insert institutional funding or computational lab acknowledgments here if applicable.]

References

[1] A. Yang et al., "Qwen2.5 Technical Report," arXiv preprint arXiv:2409.12186, 2024.

[2] F. Wright and H. Andrews, "JSON Schema: A Media Type for Describing JSON Documents," IETF Internet-Draft, Draft-07, 2020.

[3] P. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Proc. NeurIPS, 2019, pp. 8024–8035.

[4] B. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," arXiv preprint arXiv:2307.09702, 2023.

[5] G. Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in Proc. ICML, 2023, pp. 38087–38099.